

M-RAG: Semantic Key-Value Indexing for Retrieval-Augmented Generation

Xu Sun¹, Tongkai Xu¹, Baiheng Xie¹, Jun Zhang¹, Li Huang^{1,*}, Qiang Gao¹, Kunpeng Zhang²

¹School of Computing and Artificial Intelligence, Southwestern University of Finance and Economics, China

²Information Systems, University of Maryland, United States

{42123079,42311038,42211042,225085400006}@mail.swufe.edu.cn,

{lihuang,qianggao}@swufe.edu.cn, kpzhang@umd.edu

Abstract—Retrieval-augmented generation (RAG) turns external documents into evidence for large language models. In practice, this is also a data access problem: a system must decide what to index, what to retrieve, and what evidence to place in the context under a token budget. Most RAG pipelines use text chunks for both lookup and generation. This couples two different objectives. Retrieval benefits from compact and discriminative records, while generation needs contextual and faithful evidence. As a result, small chunks may fragment answer-bearing information, whereas large chunks may introduce noise and waste the context budget. We propose M-RAG, a semantic key-value indexing layer for budget-constrained RAG query processing. M-RAG extracts meta-markers from complete documents, where each record contains a retrieval key, an information value, and provenance pointers. Online retrieval operates over the key field, which can be searched by dense vector retrieval or sparse lexical retrieval; the paired values are returned as generation payloads and assembled under the token budget. Provenance pointers further support coverage validation and position-aware context ordering. This design separates the physical index entry from the evidence payload without changing the underlying retriever or generator. Experiments on LongBench QA subtasks show that M-RAG achieves competitive or better accuracy than representative chunk-based baselines, especially under tight token budgets. Further analyses show high document coverage, stronger robustness under expanding candidate corpora, and lower online retrieval latency. These results suggest that semantic key-value indexing is a practical access method for RAG workloads.

Index Terms—Retrieval-augmented generation, RAG, semantic indexing, key-value indexing.

I. INTRODUCTION

Retrieval-augmented generation (RAG) has become a common interface between large language models (LLMs) and external data. In data-intensive settings, such as enterprise knowledge bases, scientific literature search, legal analysis [1], and financial reporting [2], a RAG system depends on the relevance, completeness, and organization of the evidence it presents to the generator. This makes RAG not only a generation problem, but also a data management problem: how should evidence be represented, indexed, retrieved, and assembled before an LLM reads it?

From this perspective, RAG is an indexing and query processing problem under a token budget. Given a document collection and a user query, the system must support selective

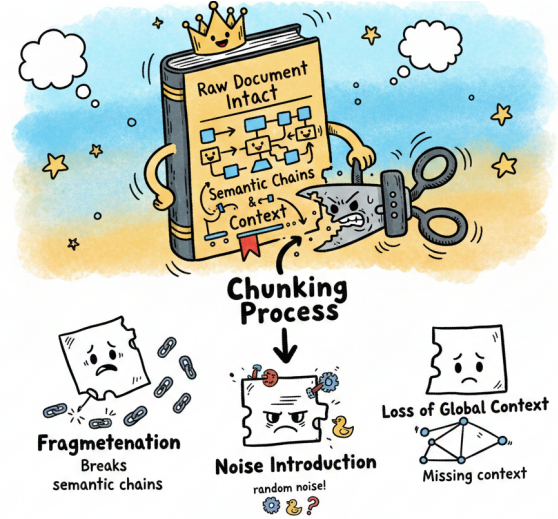


Fig. 1. An illustration of information fragmentation and structural loss caused by chunk-based RAG.

lookup, low-latency retrieval, compact context assembly, and traceability to the source data. The retrieved records are not the final output. They are intermediate evidence consumed by an LLM, whose context window is limited and whose generation can be affected by irrelevant or poorly organized context. A RAG access method therefore has to balance retrieval accuracy, evidence fidelity, provenance, and online efficiency.

Most RAG systems instantiate this access method with text chunks. A document is split into fixed-size windows, overlapping windows, semantic segments, or structure-aware spans. At query time, the retriever selects several chunks, and the selected chunks are inserted into the LLM context [3]–[6]. This design is simple, scalable, and compatible with standard vector indexes. It also makes a strong physical-design choice: the object used as the index entry is the same object returned as the generation payload.

This choice is convenient, but the two roles have different objectives. Retrieval benefits from compact and discriminative records that match a query precisely and avoid mixing unrelated topics. Generation benefits from complete and contextual evidence that preserves entities, relations, dates, discourse structure, and other answer-bearing details. A single chunk is

*Corresponding author: Li Huang (lihuang@swufe.edu.cn).

asked to serve both roles. Small chunks may improve matching but fragment evidence. Large chunks preserve more context but introduce noise, waste token budget, and enlarge the online search space. Overlap reduces some boundary errors, but it increases redundancy and keeps the same coupling between *index entry* and *payload*. Figure 1 illustrates this failure mode: chunk boundaries can split answer-bearing information and weaken the document structure seen by the retriever and generator. The issue is therefore not only where to cut a document; it is that chunking exposes a fixed access interface in which lookup representation and generation payload are tied together.

Recent work has made chunking more adaptive. Semantic chunking groups adjacent text by similarity. PIC [7] uses document summaries as pseudo-instructions for adaptive grouping. MoC [8] learns mixtures of chunking strategies. DOS RAG [9] shows that preserving the original document structure can be a strong baseline. These methods improve segmentation quality and demonstrate that the retrieval unit matters. However, they still retain the same access interface: retrieve a text span, then use that span as generation context. Long-context LLMs do not remove this design question. Although longer context windows reduce the need to use retrieval only as external memory, they still require systems to decide which evidence should enter the context, and they remain sensitive to irrelevant or distracting information [10]–[14]. Recent chunk-free retrieval work, such as CFIC [15], further suggests that raw chunks need not be the basic retrieval object.

This paper asks a simple question: should the *retrieval representation* be the same text as the *generation payload*? We argue that it should not. We propose M-RAG, a semantic key-value indexing layer for RAG. Instead of indexing raw chunks, M-RAG extracts semantic records, called meta-markers, from complete documents. Each meta-marker contains three fields: a retrieval key k , an information value v , and provenance pointers p . The key is a compact, query-facing description used for lookup. The value is a richer evidence payload used for generation. The provenance pointers connect the record back to source paragraphs. For example, a key may describe an information need such as a definition, a date, a relation, or an event, while the paired value stores the answer-bearing evidence and the provenance pointers identify where that evidence comes from.

M-RAG indexes keys and returns values. At query time, retrieval operates over the key field rather than the value field. The key can be embedded for dense retrieval or indexed as text for sparse retrieval, such as BM25. Once a key is retrieved, its paired value is assembled into the generation context under a token budget. This separates the lookup representation from the returned payload. The key can be optimized for precise matching, while the value can be optimized for evidence fidelity. The provenance pointers support coverage validation and position-aware context ordering. In this sense, M-RAG applies a familiar database principle to RAG: keep the index lightweight, and keep the payload faithful to the data needed by downstream operators.

M-RAG is designed as a modular indexing layer. Offline, an off-the-shelf LLM extracts meta-markers from tagged documents. A coverage validation step checks whether the extracted records cover the source document and creates conservative fallback records for uncovered units. Online, the system retrieves relevant keys through a dense or sparse index, selects paired values under a token budget, and orders the selected values by source position or similarity before generation. Retrieval operates over compact semantic keys; generation reads contextual evidence values.

Our contributions are:

- We formulate RAG retrieval as a semantic indexing and query processing problem, and identify the coupling between index entry and generation payload as a source of the chunk-size tradeoff.
- We introduce a semantic key-value marker representation for RAG. It separates retrieval keys, information values, and provenance pointers, allowing lookup, generation, and traceability to be handled by distinct fields.
- We design an offline construction and validation pipeline together with an online budget-aware query processing procedure for key-based retrieval and value-based context assembly.
- We evaluate M-RAG on LongBench QA subtasks. The results show competitive or higher accuracy under tight token budgets, high document coverage, and lower online retrieval latency than representative chunk-based baselines.

II. RELATED WORK

Retrieval-augmented generation (RAG) connects language models with external corpora by retrieving evidence before generation [3], [4]. The basic interface has since evolved into dense retrieval pipelines, modular RAG systems, graph-based retrieval, and agentic workflows [6], [16]–[18]. Despite these architectural differences, RAG systems share a core data-management question: what representation should be indexed, and what evidence should be returned to the generator?

Indexing and access methods for semantic data. Data systems typically separate logical records from physical access paths. In semantic search, dense-vector indexes such as HNSW [19] and sparse lexical indexes provide alternative lookup paths over a collection. This view is useful for RAG because chunking is not only preprocessing; it determines the physical record exposed to the access path. M-RAG follows this access-method perspective by materializing a record layout in which the field used for lookup is different from the payload returned to the downstream operator.

Retrieval units in RAG. Most RAG systems instantiate this choice with text spans. Fixed-size, overlapping, semantic, and structure-aware chunking methods differ in how they determine document boundaries, but they largely preserve the same access interface: a query is matched against a text unit, and the selected unit is inserted into the generation context. Prior work has shown that segmentation quality affects retrieval and downstream QA, and that the choice of granularity

creates a tension between noise and coherence [8], [20]. PIC uses document summaries as pseudo-instructions for adaptive grouping [7], while MoC learns mixtures of chunking learners and applies the learned chunkers to extract text spans [8]. Proposition-level dense retrieval studies a finer granularity by indexing concise factual propositions rather than passages or sentences [21]. These methods show that retrieval-unit design is important, but they mainly optimize the boundary or granularity of the text object being retrieved.

Structure-aware and graph-enhanced RAG. Beyond segmentation, recent work has emphasized the organization of evidence. DOS RAG preserves the original document structure and passage order as a simple retrieve-then-read baseline for long-context language models [9]. Graph-augmented methods construct entity-relation or community structures over indexed text to support retrieval beyond flat chunk collections [22], [23]. LightRAG, for example, incorporates graph structures into text indexing and retrieval and combines them with vector representations [23]. AGRAG further improves graph-based RAG by using statistics-based entity extraction and retrieving minimum-cost maximum-influence reasoning subgraphs [24]. These methods model structural dependencies in the evidence space. They are complementary to M-RAG: rather than proposing a new graph retriever or generator, M-RAG changes the logical record layout exposed to retrieval. The resulting records can still be searched with dense, sparse, or graph-enhanced backends.

Decoupled and chunk-free representations. The closest line of work relaxes the assumption that raw chunks must be the basic retrieval object. CFIC proposes chunking-free in-context retrieval by using encoded document hidden states and decoding strategies to identify evidence text for a query [15]. HeteRAG studies decoupled knowledge representations, using short chunks for generation and context-enriched multi-granular views for retrieval [25]. These studies are closely related because they also separate, to different degrees, the representation used for retrieval from the evidence consumed by generation. M-RAG takes a database access-method perspective on this design space. It materializes a persistent semantic key-value record layout: a compact query-facing key is indexed for lookup, a richer value is returned for generation, and provenance pointers keep the record traceable to source units. This layout explicitly separates query matching from evidence delivery while preserving the standard RAG generator interface.

III. METHODOLOGY

M-RAG treats RAG retrieval as an access-method design problem. A RAG system needs records that are selective enough for lookup, but also informative enough for generation. Chunk-based RAG uses the same physical text span for both roles. M-RAG separates these roles by materializing documents into semantic key-value records. As shown in Figure 2, the system constructs the records offline and, at query time, retrieves by semantic keys while returning paired evidence values to the generator.

A. Semantic Key-Value Record Model

We consider a document collection $\mathcal{D} = \{D_1, \dots, D_N\}$. Each document D_n is represented as an ordered sequence of source units,

$$D_n = (s_{n,1}, \dots, s_{n,L_n}), \quad (1)$$

where a source unit can be a paragraph or a token-based segment. Source units are not the logical retrieval records in M-RAG. They serve as the physical units used for provenance, coverage validation, and source-order reconstruction.

M-RAG materializes each document into a set of semantic key-value records, called meta-markers:

$$\mathbf{M}_{D_n} = \{m_{n,1}, \dots, m_{n,|\mathbf{M}_{D_n}|}\}, \quad (2)$$

$$m_{n,i} = (k_{n,i}, v_{n,i}, p_{n,i}). \quad (3)$$

Here $k_{n,i}$ is a query-facing key, $v_{n,i}$ is the evidence value returned for generation, and $p_{n,i} \subseteq \{1, \dots, L_n\}$ is a set of provenance pointers to source units in D_n . The key is the indexed object, the value is the returned object, and the pointer keeps the record traceable to the source document. The global marker collection is

$$\mathcal{M} = \bigcup_{n=1}^N \mathbf{M}_{D_n}. \quad (4)$$

Given a query q and a token budget B , the online task is to select a sequence of markers from $\mathcal{M}$ and assemble their values as the external context:

$$\mathbf{M}^q = (m_{\pi_1}, \dots, m_{\pi_t}), \quad (5)$$

$$\text{s.t.} \quad \sum_{j=1}^t |v_{\pi_j}| \leq B. \quad (6)$$

Only the keys k_{π_j} are used to estimate relevance. Only the values v_{π_j} are placed in the generation context:

$$G(q) = \text{LLM}_{\text{gen}}(q, v_{\pi_1} \oplus \dots \oplus v_{\pi_t}), \quad (7)$$

where $\oplus$ denotes textual concatenation. This record model separates the lookup representation from the generation payload while preserving a link to the original source units.

B. Offline Index Construction

M-RAG builds the semantic key-value index before serving queries. The construction pipeline has four steps: source-unit tagging, marker extraction, coverage validation with fallback repair, and key indexing.

Source-unit tagging. For each document D_n , every source unit $s_{n,j}$ is assigned a position identifier. These identifiers are inserted into the extractor input so that generated records can refer back to the units from which their evidence is derived. The identifiers are not used as retrieval content; they provide provenance for validation and later context ordering.

Marker extraction. Given a tagged document and an extraction instruction $\mathbb{P}$, an off-the-shelf LLM produces a set of records:

$$\mathbf{M}_{D_n} = \text{LLM}_{\text{ext}}(D_n, \mathbb{P}). \quad (8)$$

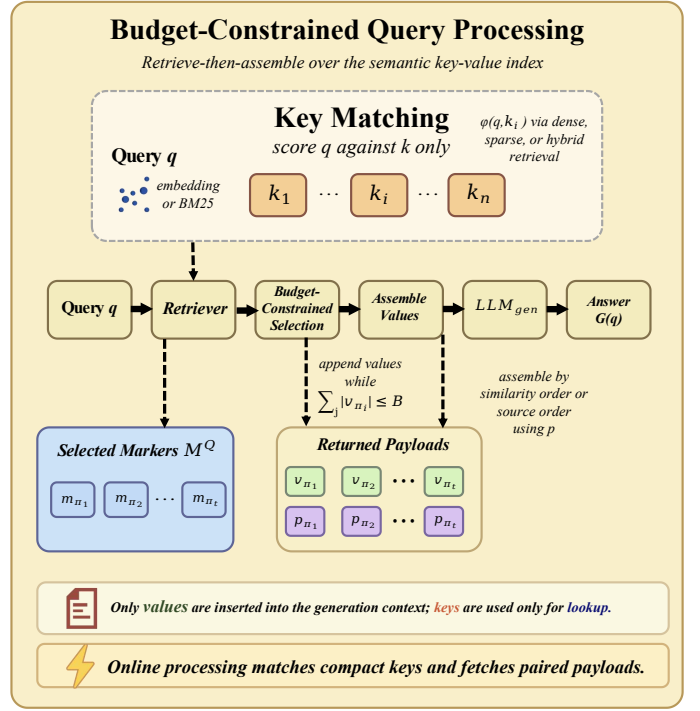
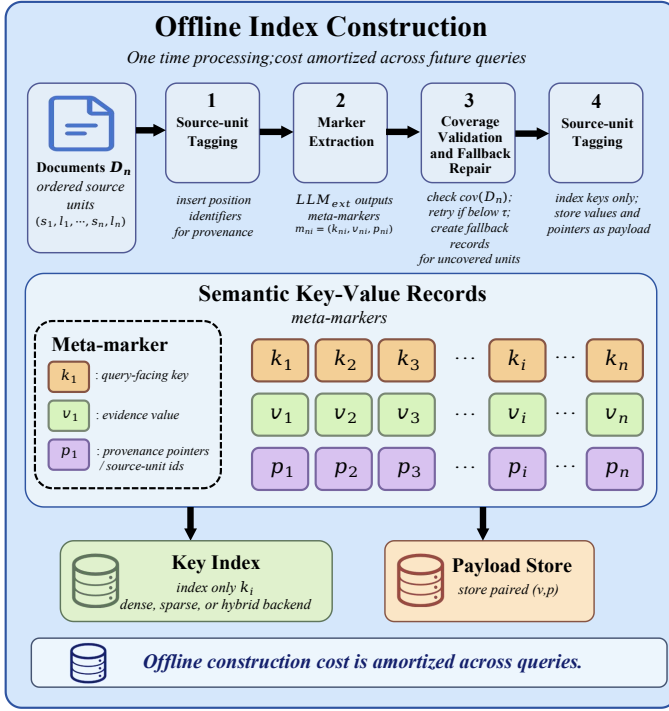


Fig. 2. The overall architecture of M-RAG.

TABLE I
PROMPT STRUCTURE FOR MARKER EXTRACTION.

Component	Role
Input	Tagged source units with position identifiers and an expected marker count derived from document length.
Output schema	A JSON-style list of records, each with key k , value v , and provenance pointer p .
Granularity	Prefer fine-grained records; each record is constrained to a small bounded set of source units.
Value field	Produce a self-contained evidence payload that preserves entities, attributes, relations, dates, and other answer-bearing details.
Key field	Produce a query-facing description that summarizes and retrieves the paired value.
Pointer field	Return source-unit identifiers used to construct the value.
Coverage	Cover all source units, allowing overlap across records; uncovered units are handled by fallback repair.

The extractor is constrained to produce records with a query-facing key, a self-contained evidence value, and bounded provenance pointers. The prompt is used only to specify this output schema and the extraction constraints; the complete prompt and implementation details are provided in the accompanying artifact. Table I summarizes the prompt structure used by M-RAG.

Coverage validation and fallback repair. Because the records are generated by an LLM, M-RAG validates the extracted index before exposing it to online retrieval. For a document D_n , provenance coverage is defined as

$$\text{cov}(D_n) = \frac{\left| \bigcup_{m \in M_{D_n}} p(m) \right|}{L_n}. \quad (9)$$

If $\text{cov}(D_n)$ is below a threshold τ , extraction is retried. If the retries still leave uncovered source units, M-RAG applies a conservative repair: each uncovered source unit is converted into a fallback record whose key and value are both the original unit content, and whose pointer refers to that unit. This repair prevents LLM extraction failures from making any source unit unreachable.

Key indexing. After validation, only keys are indexed. The values and provenance pointers are stored as payload fields attached to the corresponding records. In a dense implementation, the key set

$$K = \{k_i \mid m_i = (k_i, v_i, p_i) \in \mathcal{M}\} \quad (10)$$

is embedded by a sentence embedding model $\mathbb{E}(\cdot)$ and stored in a vector access path:

$$\mathbf{e}_{k_i} = \mathbb{E}(k_i). \quad (11)$$

In a sparse implementation, the same key strings can be indexed by a lexical retriever such as BM25. In both cases, online matching operates over keys rather than over the richer evidence values.

C. Budget-Constrained Query Processing

At query time, M-RAG executes a retrieve-then-assemble plan over the key-value index. The retrieval backend ranks records by applying a scoring function to the query and the key field:

$$\text{score}(q, m_i) = \phi(q, k_i), \quad (12)$$

where ϕ can be implemented as dense embedding similarity, lexical BM25 scoring, or another retrieval function over keys.

For dense retrieval, the query is embedded as $e_q = \mathbb{E}(q)$ and submitted to the vector access path, which returns approximate nearest-neighbor keys. For sparse retrieval, q is matched against the lexical index over key strings.

Let $(m_{\pi_1}, m_{\pi_2}, \dots)$ be the candidate sequence returned by the backend and sorted by decreasing key score. M-RAG scans this sequence and appends the paired value v_{π_j} while the cumulative value length remains within the token budget:

$$\sum_{j=1}^t |v_{\pi_j}| \leq B \quad \text{and} \quad \sum_{j=1}^{t+1} |v_{\pi_j}| > B. \quad (13)$$

The selected values can then be assembled by similarity order, which preserves the retrieval ranking, or by source order, which uses the provenance pointers to recover the document order. The assembled values form the context consumed by the generator. The retrieved keys are not inserted into the generation context unless they are also part of the stored value.

D. Design Analysis

The design changes the physical access interface of RAG. In chunk-based systems, a text span must be selective enough to match a query and complete enough to support generation. These requirements are often in tension: shorter spans improve specificity but can fragment evidence, while longer spans preserve context but mix topics and spend more of the token budget. M-RAG assigns these requirements to different fields. The key provides a compact lookup representation, while the value carries the evidence consumed downstream.

The provenance field does not participate in ranking, but it makes the semantic index auditable. Before indexing, provenance pointers support a coverage check over the source units and allow fallback repair for uncovered units. After retrieval, the same pointers support source-order assembly, which is useful when answer evidence depends on document structure rather than only on local similarity.

The main cost of this design is moved to the offline construction phase. Marker extraction materializes semantic records before queries arrive, while online processing only matches keys and fetches paired payloads. This division is suitable for read-heavy RAG workloads, where a constructed index is reused across many queries. It also keeps the retrieval backend modular: dense, sparse, or hybrid retrievers can operate over the same key field without changing the generator interface.

IV. EXPERIMENTS

A. Experimental Setup

Benchmark. We evaluate M-RAG on QA subtasks from LongBench [26], a benchmark for long-context understanding. The evaluation uses NarrativeQA [27] and Qasper [28] for single-hop QA, and 2WikiMultihopQA [29] for multi-hop QA. Each subtask contains 200 samples and covers contexts longer than 3K tokens on average, spanning narrative, scientific, and multi-document reasoning workloads.

Baselines. We compare M-RAG with representative retrieval-unit construction methods. The chunk-based baselines include

Fixed-Size RAG with 128-token segments, Fixed-Size RAG with overlap, Semantic RAG [5], PIC RAG [7], and DOS RAG [9]. These methods address the same physical-design question as M-RAG: how evidence should be organized before retrieval and generation. We also include LightRAG [23] as a graph-enhanced RAG baseline that combines graph structures with text retrieval.

Retrieval and generation. We evaluate each method under two retrieval backends. For dense retrieval, we use BAAI/bge-m3 [30]. For sparse retrieval, we use BM25 over each method’s retrieval field: semantic keys for M-RAG and text chunks for chunk-based baselines. All methods are evaluated under the same retrieved-evidence budgets of 128×1 , 128×3 , and 128×5 tokens. Generation is performed by Qwen3-30B-A3B-Instruct-2507 [31] with the same answer-generation prompt across methods. The generator receives the selected information values for M-RAG and the retrieved text chunks for chunk-based baselines.

M-RAG configuration. Marker extraction uses DeepSeek-V3.2 [32] with one prompt template across all benchmarks. We report zero-shot and few-shot extraction variants; in the few-shot setting, the extractor receives one benchmark-specific marker example following the same key-value-pointer schema. We also evaluate two context assembly orders: position sorting (-P), which orders values by provenance position, and similarity sorting (-S), which preserves retrieval ranking. Both the extractor and generator use temperature 0. Each experiment is repeated five times, and we report mean and standard deviation. Experiments are conducted on a server with NVIDIA A800 GPUs (80GB). The implementation, prompts, few-shot examples, and reproduction artifacts are available at <https://github.com/9sxx/M-RAG>.

B. End-to-End QA Performance

Table II reports end-to-end QA performance on three LongBench subtasks under two retrieval backends and three evidence budgets. Across the 18 dataset–retriever–budget settings, at least one M-RAG variant ranks first or second in 16 settings. The gains are most visible under tight budgets, where the retrieved unit has a direct effect on whether the generator receives answer-bearing evidence. With BAAI/bge-m3 at 128×1 , M-RAG ranks first on Qasper and 2WikiMultihopQA and second on NarrativeQA. With BM25, M-RAG achieves the best score on NarrativeQA and Qasper at 128×1 and becomes best on 2WikiMultihopQA at 128×3 .

The results also show that the benefit is not tied to a single retrieval backend. Dense retrieval benefits from matching queries against compact semantic keys, while BM25 benefits from a concise lexical field that avoids term dilution in long chunks. This consistency supports the main design choice of M-RAG: retrieval should operate over a query-facing representation, while the generation budget should be spent on the paired evidence value.

The advantage is smaller when the evidence budget increases. Strong span-based methods such as PIC and DOS can recover more context at larger budgets and lead in several

TABLE II

PERFORMANCE COMPARISON IN F1 SCORE FOLLOWING LONGBENCH. RESULTS ARE GROUPED BY DATASET. FOR EACH METHOD, WE REPORT PERFORMANCE UNDER BOTH **BAAI/BGE-M3** AND **BM25** RETRIEVERS ACROSS THREE RETRIEVAL BUDGETS (128×1 / 128×3 / 128×5). BEST RESULT IN **BOLD**, SECOND BEST UNDERLINED. M-RAG REPORTS ZERO-SHOT AND FEW-SHOT VARIANTS WITH POSITION SORTING (-P) AND SIMILARITY SORTING (-S).

	BAAI/bge-m3			BM25		
	128×1	128×3	128×5	128×1	128×3	128×5
NarrativeQA						
Fixed-Size	0.0660 ± 0.0000	0.0952 ± 0.0002	0.1094 ± 0.0001	0.0568 ± 0.0015	0.0920 ± 0.0032	0.1178 ± 0.0009
w. overlap	0.0581 ± 0.0000	0.0906 ± 0.0000	0.0982 ± 0.0001	0.0545 ± 0.0015	0.0877 ± 0.0050	0.1201 ± 0.0040
Semantic	0.0617 ± 0.0000	0.1200 ± 0.0002	0.1449 ± 0.0009	0.0574 ± 0.0015	0.0940 ± 0.0029	0.1102 ± 0.0016
PIC	0.0618 ± 0.0009	0.1092 ± 0.0001	0.1226 ± 0.0005	0.0565 ± 0.0030	0.0962 ± 0.0029	0.1148 ± 0.0086
DOS	0.0843 ± 0.0000	0.1191 ± 0.0000	0.1398 ± 0.0001	0.0434 ± 0.0038	0.0926 ± 0.0042	0.1146 ± 0.0033
LightRAG	0.0614 ± 0.0015	0.1051 ± 0.0076	0.1201 ± 0.0042	0.0027 ± 0.0010	0.0412 ± 0.0037	0.0700 ± 0.0046
M-RAG-P	0.0684 ± 0.0006	0.1279 ± 0.0018	<u>0.1459 ± 0.0022</u>	0.0572 ± 0.0041	0.0940 ± 0.0007	0.1176 ± 0.0045
M-RAG-S	0.0657 ± 0.0025	<u>0.1214 ± 0.0014</u>	0.1476 ± 0.0031	0.0585 ± 0.0013	0.0984 ± 0.0019	0.1183 ± 0.0040
M-RAG[†]-P	<u>0.0736 ± 0.0029</u>	0.1247 ± 0.0030	0.1381 ± 0.0018	0.0771 ± 0.0039	0.1078 ± 0.0053	0.1147 ± 0.0024
M-RAG[†]-S	0.0644 ± 0.0004	0.1208 ± 0.0029	0.1357 ± 0.0013	<u>0.0757 ± 0.0014</u>	<u>0.1057 ± 0.0025</u>	<u>0.1197 ± 0.0015</u>
Qasper						
Fixed-Size	0.1316 ± 0.0000	0.1908 ± 0.0000	0.2425 ± 0.0006	0.1043 ± 0.0037	0.1693 ± 0.0081	0.2221 ± 0.0015
w. overlap	0.1294 ± 0.0003	0.1963 ± 0.0002	0.2361 ± 0.0014	0.1026 ± 0.0076	0.2032 ± 0.0080	0.2257 ± 0.0067
Semantic	0.1654 ± 0.0008	0.2482 ± 0.0007	0.2683 ± 0.0003	0.1297 ± 0.0102	0.1769 ± 0.0034	0.2267 ± 0.0032
PIC	0.1581 ± 0.0006	0.2530 ± 0.0036	0.2838 ± 0.0021	0.1230 ± 0.0075	0.1939 ± 0.0046	0.2341 ± 0.0066
DOS	0.1643 ± 0.0009	0.2269 ± 0.0009	0.2635 ± 0.0024	0.1142 ± 0.0038	0.1798 ± 0.0036	0.2462 ± 0.0064
LightRAG	0.0858 ± 0.0010	0.1761 ± 0.0050	0.1928 ± 0.0051	0.0362 ± 0.0016	0.0761 ± 0.0038	0.1344 ± 0.0062
M-RAG-P	0.1436 ± 0.0018	0.2317 ± 0.0013	0.2456 ± 0.0013	0.1364 ± 0.0059	0.2047 ± 0.0031	0.2268 ± 0.0054
M-RAG-S	0.1390 ± 0.0037	0.2308 ± 0.0032	0.2655 ± 0.0055	0.1159 ± 0.0030	0.2008 ± 0.0164	0.2404 ± 0.0053
M-RAG[†]-P	0.1806 ± 0.0043	0.2370 ± 0.0037	0.2693 ± 0.0061	0.1174 ± 0.0018	<u>0.2038 ± 0.0054</u>	0.2389 ± 0.0029
M-RAG[†]-S	0.1820 ± 0.0025	0.2224 ± 0.0041	0.2690 ± 0.0015	<u>0.1298 ± 0.0025</u>	0.2026 ± 0.0068	<u>0.2412 ± 0.0020</u>
2WikiMultihopQA						
Fixed-Size	0.2220 ± 0.0000	0.3239 ± 0.0038	0.3832 ± 0.0023	0.2008 ± 0.0153	0.2906 ± 0.0096	0.3231 ± 0.0161
w. overlap	0.1792 ± 0.0000	0.3186 ± 0.0004	0.3740 ± 0.0018	0.1793 ± 0.0115	0.2898 ± 0.0062	0.3058 ± 0.0040
Semantic	0.1875 ± 0.0005	0.3125 ± 0.0030	0.3831 ± 0.0013	0.1693 ± 0.0125	0.2881 ± 0.0184	0.3312 ± 0.0057
PIC	0.2066 ± 0.0000	0.3236 ± 0.0015	0.3947 ± 0.0026	0.1610 ± 0.0084	0.2886 ± 0.0111	0.2993 ± 0.0100
DOS	0.2023 ± 0.0000	0.3255 ± 0.0004	0.4110 ± 0.0028	0.1908 ± 0.0021	0.2991 ± 0.0200	0.3245 ± 0.0123
LightRAG	0.1308 ± 0.0077	0.2304 ± 0.0040	0.2687 ± 0.0057	0.0604 ± 0.0062	0.1063 ± 0.0042	0.1756 ± 0.0047
M-RAG-P	0.2134 ± 0.0016	0.3229 ± 0.0030	0.3494 ± 0.0057	0.1657 ± 0.0116	0.3042 ± 0.0080	0.3372 ± 0.0096
M-RAG-S	0.2170 ± 0.0077	0.3237 ± 0.0069	0.3582 ± 0.0035	0.1680 ± 0.0083	0.2906 ± 0.0054	<u>0.3342 ± 0.0037</u>
M-RAG[†]-P	0.2232 ± 0.0039	0.3301 ± 0.0003	0.3948 ± 0.0006	0.1606 ± 0.0069	0.2745 ± 0.0060	0.2835 ± 0.0028
M-RAG[†]-S	0.2321 ± 0.0015	0.3290 ± 0.0000	0.3560 ± 0.0006	0.1650 ± 0.0024	0.2842 ± 0.0087	0.3131 ± 0.0063

[†] denotes few-shot marker extraction; unmarked M-RAG rows use zero-shot marker extraction.

high-budget settings, including Qasper under dense retrieval and 2WikiMultihopQA under dense retrieval at 128×5 . These exceptions are informative rather than contradictory: chunks can work well when enough context can be returned, but M-RAG is strongest when evidence selection is constrained. We therefore first inspect the constructed semantic index, then analyze its retrieval behavior and robustness.

C. Semantic Index Analysis

Document Coverage. Table III reports the provenance coverage of extracted markers. Both zero-shot and few-shot extraction cover more than 99.8% of source units on average across the three benchmarks, while fallback records account for less than 1% of the units. This suggests that LLM-based marker extraction does not introduce systematic coverage gaps under the evaluated settings. Figure 3 shows a fallback example from Qasper₅₄ [33]: the uncovered unit comes from a conclusion

paragraph with little standalone semantic content. Few-shot extraction slightly reduces fallback usage in NarrativeQA and Qasper, but the zero-shot setting already provides comparable mean coverage, indicating that the schema-level extraction instruction transfers across the evaluated workloads.

TABLE III
STATISTICS ON AVERAGE COVERAGE RATES UNDER ZERO-SHOT (ZS) AND FEW-SHOT (FS) PROMPTING.

Benchmark	Prompting	Avg. Cover.	Variance	Fallback (%)
NarrativeQA	ZS	99.80	0.4040	0.7
	FS	99.87	0.1948	0.2
Qasper	ZS	99.93	0.2316	0.9
	FS	99.90	0.2776	0.1
2WikiMultihopQA	ZS	99.80	0.5379	0.0
	FS	99.84	0.3648	0.5

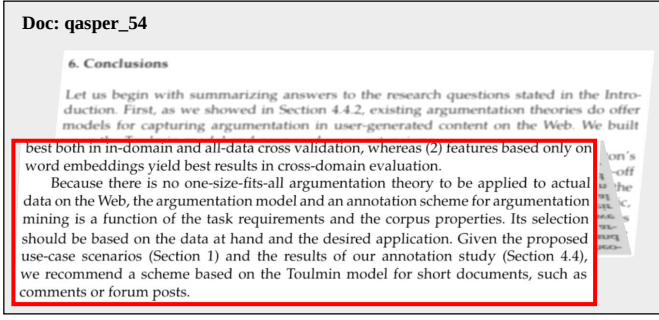


Fig. 3. A fallback example from document Qasper_54. Red box highlights the retrieved content.

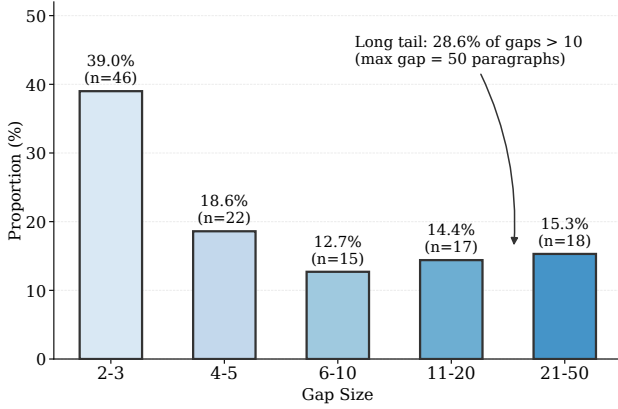


Fig. 4. Distribution of paragraph index gaps in non-consecutive multi-index markers.

Marker Granularity. Table IV shows that most markers correspond to single-paragraph evidence, indicating alignment with localized discourse units. At the same time, 29.7–36.5% of markers span continuous multi-paragraph regions, which allows a record to aggregate evidence that crosses a paragraph boundary. Non-consecutive markers are rare, suggesting that M-RAG mostly preserves the local structure of the source document while still allowing occasional long-range aggregation.

TABLE IV
PARAGRAPH INDEX DISTRIBUTION OF EXTRACTED MARKERS.

Benchmark	Total	Single	Multi-Paragraph	
		Paragraph	Continuous	Non-consec.
NarrativeQA	61,632	40,755 (66.1%)	20,849 (33.8%)	28 (0.05%)
2WikiMultihopQA	16,345	11,459 (70.1%)	4,856 (29.7%)	30 (0.18%)
Qasper	9,370	5,888 (62.8%)	3,422 (36.5%)	60 (0.64%)

For the small number of non-consecutive markers, Figure 4 shows the distribution of paragraph gaps. Many gaps remain local, but the distribution has a long tail: 28.6% of gaps exceed 10 paragraphs, and the maximum span reaches 50 paragraphs. These cases show that marker extraction can occasionally group distant but semantically related evidence, a pattern that fixed local windows are unlikely to capture directly.

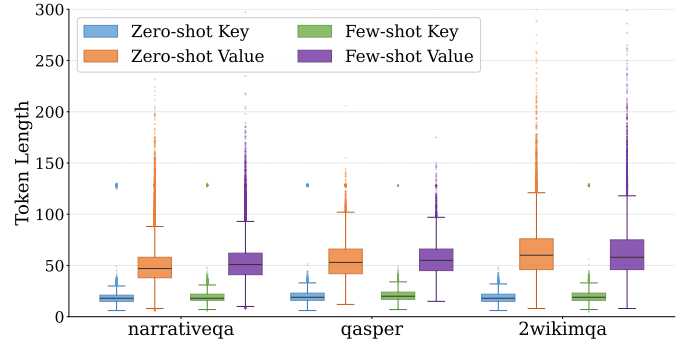


Fig. 5. Token length of k, v across benchmarks, where boxes show quartiles, whiskers extend to 1.5 IQR, and outliers are marked.

Token Length of k, v . Figure 5 presents the token length statistics of *retrieval keys* and *information values* under zero-shot (ZS) and few-shot (FS) prompting. Across all benchmarks and prompting regimes, *retrieval keys* remain compact, averaging 19–20 tokens with low variance. *Information values* are longer, averaging 50–65 tokens, or about 2.5–3 $\times$ the length of keys. This length asymmetry is consistent with the intended key-value separation: k serves as a lightweight retrieval representation, while v carries more context for generation. The value length also varies by workload. NarrativeQA tends to produce shorter values, Qasper requires longer explanatory values, and 2WikiMultihopQA yields the longest values, consistent with its multi-hop and cross-document evidence. The compactness of k remains stable across these workloads and across ZS/FS prompting.

D. Retrieval Behavior

This section examines how the semantic key-value index changes the retrieval space and the evidence returned to the generator.

Retrieval Geometry and Case Study. We conduct case studies using sampled documents from each benchmark. Figure 6(a) visualizes the query and corresponding markers using t-SNE. A consistent pattern emerges: *retrieval keys* form localized, query-aligned clusters, whereas *information values* occupy a broader region of the embedding space. This separation holds across benchmarks, including 2WikiMultihopQA where multi-hop reasoning is inherently complex. Few-shot prompting further tightens key clusters, consistent with the reduced variance observed in token length analysis. Figure 6(b) presents a Qasper example involving a definition query about “robustness”. The desired evidence is a sentence-level conceptual statement, not a broader procedural description. Fixed-Size and Semantic retrieval return information-rich passages, but these chunks mix several themes, including regularization, examples, and reference details, and do not directly state the definition. One plausible reason is that chunk embeddings represent blended topics when a span contains multiple semantic units. In contrast, meta-marker retrieval matches the query to an intent-aligned key and returns a

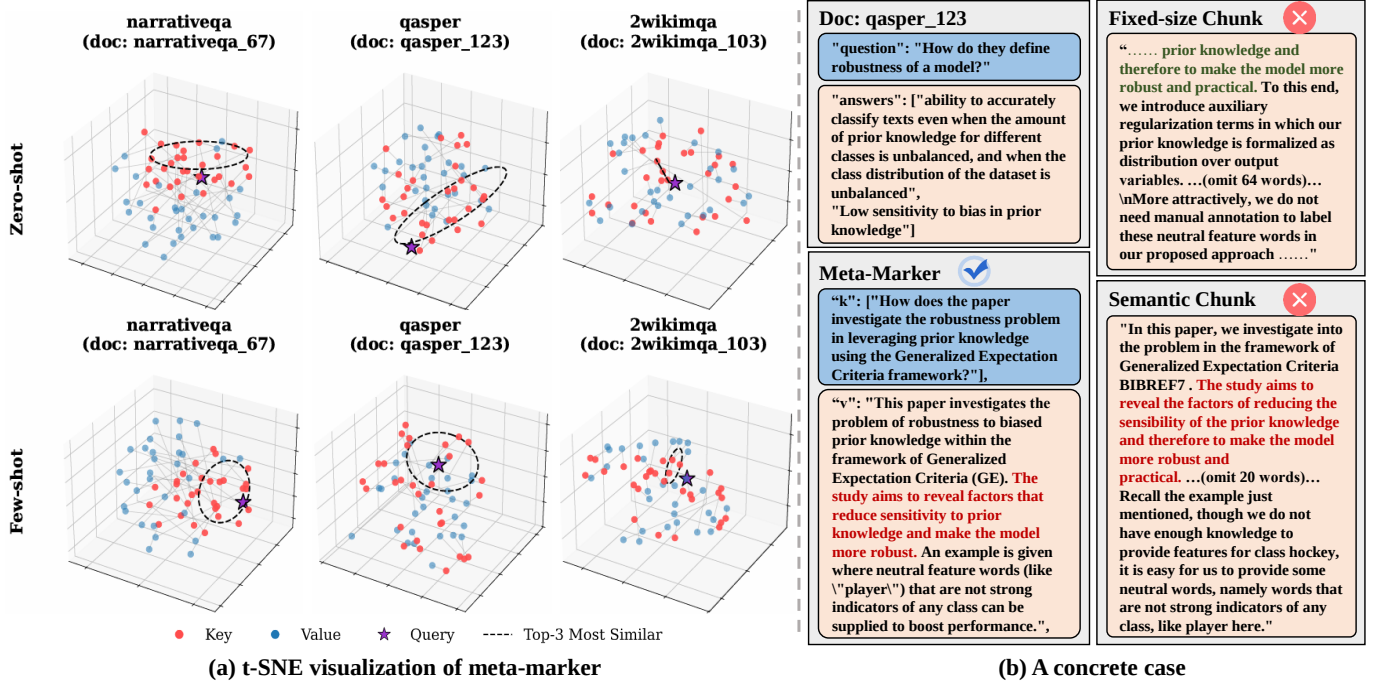


Fig. 6. Retrieval behavior of M-RAG. The visualizations show query-aligned key neighborhoods and broader value distributions; the Qasper case illustrates how key-based lookup returns a definition-aligned value while chunk-based retrieval mixes multiple themes.

value that explicitly encodes the definitional statement, as highlighted in Figure 6.

Effect of Key-Value Decomposition. These observations help explain the behavior of M-RAG in the main results. At the retrieval stage, compact *retrieval keys* provide a focused lookup field rather than forcing the retriever to search over heterogeneous generation payloads. In dense retrieval, this focus is reflected in the tighter neighborhoods shown in Figure 6(a). In sparse retrieval, the same key field exposes concise lexical evidence for BM25 matching, reducing term dilution from long chunks that mix multiple topics. At the generation stage, the length asymmetry between *retrieval keys* and *information values* allows v to retain contextual evidence, while k remains lightweight for efficient lookup. This index-data separation follows a common information-systems principle: the object used for lookup does not need to be identical to the payload returned to the downstream operator. We next test whether this focused lookup field remains selective when the candidate corpus grows.

E. Retrieval Robustness

To evaluate the robustness of M-RAG under expanding retrieval spaces, we conduct a controlled corpus-size stress test. For each query, we fix the query, answer generation model, retrieval budget, and the corresponding gold document, while progressively increasing the number of candidate indexed documents available to the retriever. The candidate corpus consists of the gold document together with deterministically sampled distractor documents from the same benchmark. Corpus size

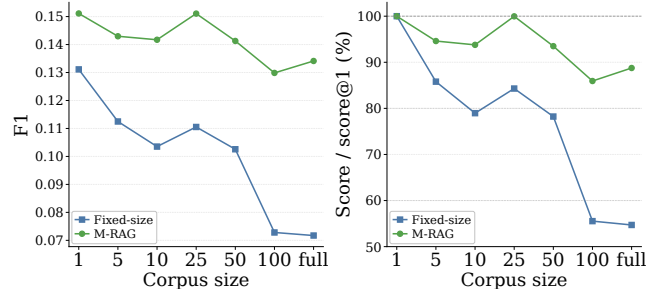


Fig. 7. Robustness under expanding candidate corpora on NarrativeQA. M-RAG retains a larger fraction of its single-document performance as distractor documents are added.

is varied from 1 to 5, 10, 25, 50, 100, and finally the full corpus.

Figure 7 reports both the absolute F1 score and the performance retention under different corpus sizes. The retention ratio is defined as the F1 score at a given corpus size normalized by the F1 score when only the gold document is available. As the retrieval space expands from a single document to the full corpus, M-RAG retains performance better than the fixed-size dense retrieval baseline. On NarrativeQA, the F1 score of M-RAG decreases from 0.1511 to 0.1341 as the retrieval space expands from a single document to the full corpus, corresponding to a retention ratio of 88.8%. In contrast, the fixed-size baseline drops from 0.1311 to 0.0717, retaining only 54.7% of its original performance.

These results indicate that M-RAG is less sensitive to distractor documents in this stress test. As corpus size grows, the gap between M-RAG and the fixed-size baseline widens, suggesting that compact semantic keys provide a more selective access path when the retriever must search over a larger candidate space. We then examine whether this behavior depends on prompt wording or context assembly order.

F. Design Sensitivity

Prompt Sensitivity. To check whether marker extraction depends on a particular prompt wording, we evaluate three prompt variants on Qasper under few-shot position sorting. Prompt A is the original extraction prompt, Prompt B swaps the output-format and critical-requirement sections, and Prompt C moves the key-goal section below the critical-requirement section. Table V shows that the results remain close across budgets, suggesting that extraction mainly depends on the record schema and coverage constraints rather than a single prompt ordering.

TABLE V
PROMPT SENSITIVITY ON QASPER UNDER FEW-SHOT POSITION SORTING.

Token Budget	Prompt A	Prompt B	Prompt C
128×1	0.1806 ± 0.0043	0.1866 ± 0.0004	0.1734 ± 0.0003
128×2	0.2210 ± 0.0018	0.2226 ± 0.0006	0.2105 ± 0.0022
128×3	0.2370 ± 0.0037	0.2370 ± 0.0004	0.2318 ± 0.0000
128×4	0.2612 ± 0.0056	0.2534 ± 0.0009	0.2571 ± 0.0010
128×5	0.2693 ± 0.0061	0.2560 ± 0.0019	0.2711 ± 0.0006

Sorting Strategies. To examine how context assembly affects M-RAG, we compare position-based sorting (PS) and similarity-based sorting (SS) under zero-shot and few-shot marker extraction. Figure 8 reports F1 scores across the three benchmarks and five retrieval budgets. Position sorting is generally stronger on structure-sensitive workloads. In 2Wiki-MultihopQA, PS improves the average score over SS by 0.9%, with few-shot PS reaching 39.48% at 640 tokens compared with 35.60% for SS. A similar pattern appears in Qasper at mid-to-high budgets, where preserving document order helps maintain the structure of technical evidence.

Similarity sorting is more useful for open-ended narrative queries. On NarrativeQA, zero-shot SS reaches 14.76% at 640 tokens, a 1.2% gain over PS. This suggests that, when evidence is less tied to document order, prioritizing query relevance during assembly can be more effective. The two sorting policies therefore act as alternative execution choices over the same semantic key-value index: PS favors source structure, while SS favors local query relevance.

G. Efficiency and Cost

This section reports online matching time, the efficiency comparison with LightRAG, and the offline costs associated with marker extraction and index maintenance.

Online Retrieval Efficiency. Figure 9 compares online similarity matching time between user queries and retrieval units. M-RAG has the lowest retrieval latency across all benchmarks. Since retrieval is performed over precomputed dense vectors

with the same embedding dimensionality, the difference is not simply a result of shorter text strings at query time. A likely contributing factor is that query-like keys form a more focused retrieval space than raw chunks, whose embeddings may mix several topics.

Index Construction Efficiency. Figure 10 compares the overall efficiency of LightRAG and M-RAG. Beyond online retrieval latency, we additionally evaluate the cost of offline index construction. Despite generating both retrieval keys and information values, M-RAG requires approximately 190 seconds per document for index construction, compared with about 460 seconds per document for LightRAG. M-RAG also has lower retrieval latency across all retrieval budgets in this comparison. Even under a retrieval budget of 5×128 tokens, retrieval latency remains below 0.2 seconds, whereas LightRAG requires several seconds per query. These results show that, relative to this graph-enhanced baseline, semantic key-value indexing offers a lower-cost offline construction path and faster online retrieval.

To further quantify the LLM overhead during index construction, we compare the indexing cost of M-RAG with LightRAG using its default indexing configuration as the baseline. The reported statistics include only the LLM calls required for offline index construction, excluding embedding computation, retrieval, and answer generation. As shown in Fig. 10, M-RAG requires only 525 LLM invocations, compared with 5,507 for LightRAG, representing a $10.5\times$ reduction in LLM invocations. Although omitted from the figure for clarity, M-RAG consumes only 7.77M input/output tokens, compared with 42.42M for LightRAG, reducing the estimated API cost from \$7.39 to \$1.59, corresponding to a $4.65\times$ reduction in indexing cost. This improvement stems from M-RAG’s document-level semantic marker extraction, which generates semantic markers once per document, whereas LightRAG repeatedly performs chunk-level entity extraction across the corpus.

Amortized Offline Cost. Practical deployment of M-RAG requires considering the cost of LLM-based marker extraction, but this cost belongs to the offline index-construction phase rather than online query processing. Using the DeepSeek-V3.2 API with *Context Caching*, processing 16.1M tokens costs about \$0.56 per 1M tokens ($\approx$ \$0.053 per 1,000 markers). Although sequential extraction takes about 17.3 minutes per document, marker extraction is naturally parallel across documents; with moderate concurrency, e.g., 100 requests, generating 1M output tokens requires 0.209 hours.

Because marker extraction is performed once during offline index construction, its cost is amortized over all subsequent queries served by the same index. Let C_{ext} denote the one-time LLM-based marker extraction cost, C_{idx} the one-time key embedding and indexing cost, and C_{ret} the online retrieval and value lookup cost per query. For Q queries over a fixed index, the effective non-generation cost per query is

$$C_{\text{eff}}(Q) = \frac{C_{\text{ext}} + C_{\text{idx}}}{Q} + C_{\text{ret}}. \quad (14)$$

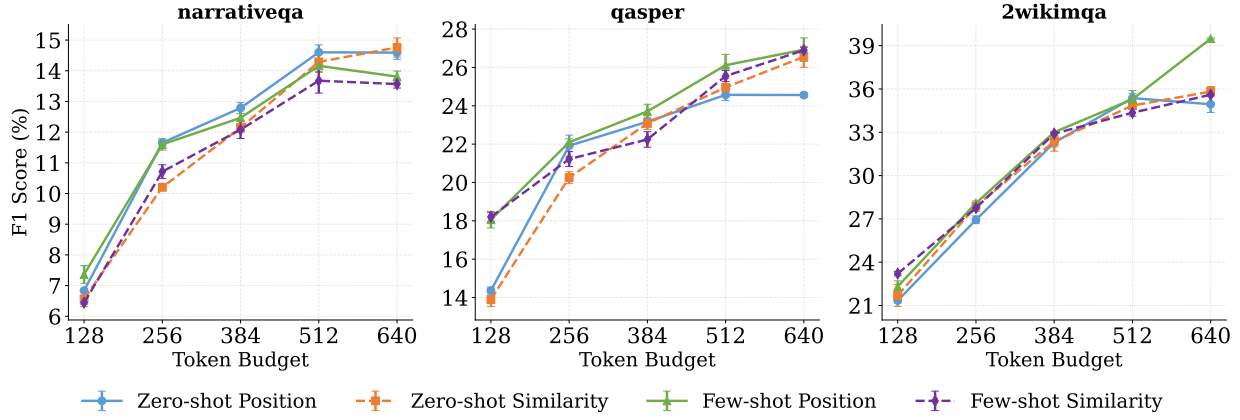


Fig. 8. Performance comparison of sorting strategies and prompting regimes across benchmarks (token budgets: 128×1 , 128×2 , 128×3 , 128×4 , 128×5).

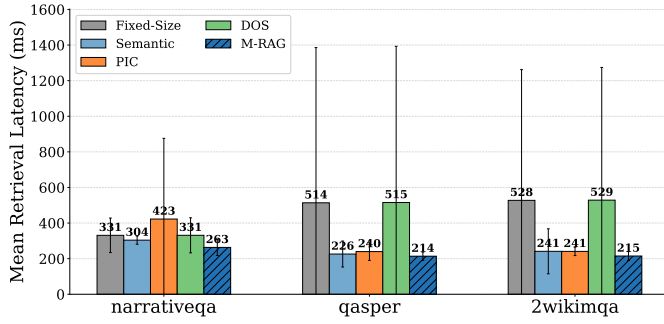


Fig. 9. Online retrieval latency across benchmarks. M-RAG matches queries against compact semantic keys and achieves lower matching time than chunk-based retrieval units.

As Q increases, the amortized offline term decreases monotonically and the effective cost approaches the online retrieval cost, i.e., $\lim_{Q \rightarrow \infty} C_{\text{eff}}(Q) = C_{\text{ret}}$. This formulation captures the write-once, read-many behavior of M-RAG without requiring a separate amortization experiment.

Incremental Updates for Dynamic Knowledge Bases. For dynamic knowledge bases, M-RAG does not need to rebuild the entire marker index when new documents arrive. Instead, only the newly added documents are processed to extract markers, embed keys, and append entries to the existing index. If the existing collection contains N documents and an update adds ΔN documents, full rebuilding scales with $N + \Delta N$, whereas incremental maintenance scales only with ΔN :

$$C_{\text{update}}^{\text{inc}} = O(\Delta N), \quad C_{\text{update}}^{\text{full}} = O(N + \Delta N). \quad (15)$$

This follows directly from the independence of offline marker extraction across documents: previously extracted markers and paired values remain unchanged, while new entries can be appended to the key-value index.

H. Limitations

Table VI illustrates a failure case of M-RAG. The k - v decomposition preserves the answer in the *information value*, but retrieval can still fail when the *retrieval key* is too coarse

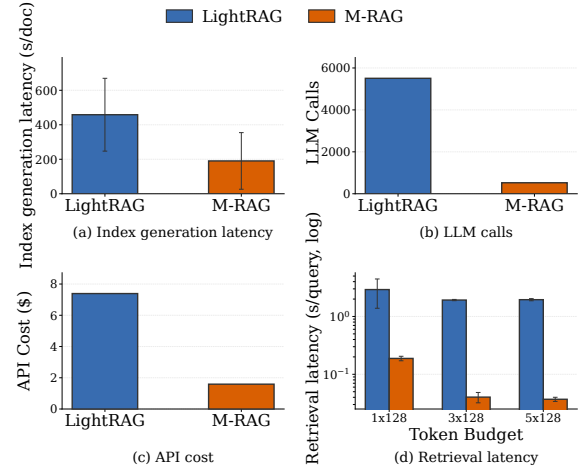


Fig. 10. Efficiency comparison between LightRAG and M-RAG. (a) Index generation latency per document. (b) LLM calls required for index construction. (c) Estimated API cost for index construction. (d) Retrieval latency under different retrieval budgets. Error bars denote standard deviation where applicable.

TABLE VI
A FAILURE CASE FROM 2WIKIMULTIHOPQA.

Case Study: Granularity Mismatch

Query: When is Henrietta Maria Of Brandenburg-Schwedt’s father’s birthday?

Retrieval Key (k): What were Philip William’s titles and his role as governor?

Information Value (v): Philip William, Prince in Prussia (19 May 1669–19 December 1711) was the first owner of the Prussian secundogeniture...

for the query. In this example, the key summarizes Philip William’s titles and governance roles, whereas the query asks for a specific relational attribute: the father’s birthday. Although the correct date appears in the value, the key does not expose that relation clearly enough, weakening the q - k match during retrieval. This limitation is most likely for fine-grained relational questions, where a generated key may summarize

an entity or event without naming the exact relation needed by the query. The failure therefore comes from query–key misalignment rather than loss of source evidence.

V. CONCLUSION

This paper studies RAG as a data access problem under a token budget. We show that chunking couples two different roles: the record used for lookup and the evidence payload consumed by the generator. M-RAG addresses this coupling by materializing documents into semantic key-value records, where compact retrieval keys support dense or sparse lookup, information values provide faithful generation evidence, and provenance pointers make the index traceable and order-aware. The index is constructed offline with coverage validation and fallback repair, and queried online by retrieving keys and assembling paired values within the context budget. Experiments on LongBench QA subtasks show that M-RAG is especially effective when evidence budgets are tight, while further analyses show high source coverage, robustness to larger candidate corpora, and lower retrieval latency. These results support a simple design principle: the object used for lookup need not be the object returned for generation. Semantic key-value indexing therefore provides a modular access layer for RAG systems, leaving room for stronger key refinement, provenance-aware execution plans, and incremental index maintenance.

AI-GENERATED CONTENT ACKNOWLEDGEMENT

In the preparation of this manuscript, we used Claude Code to assist with programming, Nano Banana [34] to generate Figure 1, and ChatGPT for grammar polishing. The authors reviewed and verified all AI-assisted content and remain responsible for the correctness and originality of the manuscript.

REFERENCES

- [1] A. Louis, G. van Dijck, and G. Spanakis, “Interpretable long-form legal question answering with retrieval-augmented large language models,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, 2024, pp. 22 266–22 275.
- [2] B. Zhang, H. Yang, T. Zhou, M. Ali Babar, and X.-Y. Liu, “Enhancing financial sentiment analysis via retrieval augmented large language models,” in *Proceedings of the fourth ACM international conference on AI in finance*, 2023, pp. 349–356.
- [3] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33. Curran Associates, Inc., 2020, pp. 9459–9474.
- [4] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, “Realm: retrieval-augmented language model pre-training,” in *Proceedings of the 37th International Conference on Machine Learning*, ser. ICML’20. JMLR.org, 2020.
- [5] G. Kamradt, “Semantic chunking,” 2024.
- [6] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, “Retrieval-augmented generation for large language models: A survey,” 2024. [Online]. Available: <https://arxiv.org/abs/2312.10997>
- [7] Z. Wang, C. Gao, C. Xiao, Y. Huang, S. Si, K. Luo, Y. Bai, W. Li, T. Duan, C. Lv, G. Lu, G. Chen, F. Qi, and M. Sun, “Document segmentation matters for retrieval-augmented generation,” in *Findings of the Association for Computational Linguistics: ACL 2025*, W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, Eds. Vienna, Austria: Association for Computational Linguistics, Jul. 2025, pp. 8063–8075. [Online]. Available: <https://aclanthology.org/2025.findings-acl.422/>
- [8] J. Zhao, Z. Ji, Z. Fan, H. Wang, S. Niu, B. Tang, F. Xiong, and Z. Li, “MoC: Mixtures of text chunking learners for retrieval-augmented generation system,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, Eds. Vienna, Austria: Association for Computational Linguistics, Jul. 2025, pp. 5172–5189. [Online]. Available: <https://aclanthology.org/2025.acl-long.258/>
- [9] A. Laitenberger, C. D. Manning, and N. F. Liu, “Stronger baselines for retrieval-augmented generation with long-context language models,” in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, Eds. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 32 547–32 557. [Online]. Available: <https://aclanthology.org/2025.emnlp-main.1656/>
- [10] C.-P. Hsieh, S. Sun, S. Krizan, S. Acharya, D. Rekesh, F. Jia, and B. Ginsburg, “RULER: What’s the real context size of your long-context language models?” in *First Conference on Language Modeling*, 2024. [Online]. Available: <https://openreview.net/forum?id=kloBbc76Sy>
- [11] D. Wang, Q. Huang, M. Jackson, and J. Gao, “Retrieve what you need: A mutual learning framework for open-domain question answering,” *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 247–263, 2024. [Online]. Available: <https://aclanthology.org/2024.tacl-1.14/>
- [12] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, “Self-RAG: Learning to retrieve, generate, and critique through self-reflection,” in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=hSyW5go0v8>
- [13] C. Nuengsigkapijan, “Hifi-rag: Hierarchical content filtering and two-pass generation for open-domain rag,” 2025. [Online]. Available: <https://arxiv.org/abs/2512.22442>
- [14] D. Wang, J. Ma, and S. Kumar, “Retrieval augmented question answering: When should llms admit ignorance?” 2025. [Online]. Available: <https://arxiv.org/abs/2512.23836>
- [15] H. Qian, Z. Liu, K. Mao, Y. Zhou, and Z. Dou, “Grounding language model with chunking-free in-context retrieval,” in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, L.-W. Ku, A. Martins, and V. Srikumar, Eds. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 1298–1311. [Online]. Available: <https://aclanthology.org/2024.acl-long.71/>
- [16] W. Fan, Y. Ding, L. Ning, S. Wang, H. Li, D. Yin, T.-S. Chua, and Q. Li, “A survey on rag meeting llms: Towards retrieval-augmented large language models,” in *Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining*, 2024, pp. 6491–6501.
- [17] A. Singh, A. Ehtesham, S. Kumar, and T. T. Khoei, “Agentic retrieval-augmented generation: A survey on agentic rag,” 2025. [Online]. Available: <https://arxiv.org/abs/2501.09136>
- [18] Y. Li, W. Zhang, Y. Yang, W.-C. Huang, Y. Wu, J. Luo, Y. Bei, H. P. Zou, X. Luo, Y. Zhao, C. Chan, Y. Chen, Z. Deng, Y. Li, H.-T. Zheng, D. Li, R. Jiang, M. Zhang, Y. Song, and P. S. Yu, “A survey of RAG-reasoning systems in large language models,” in *Findings of the Association for Computational Linguistics: EMNLP 2025*, C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, Eds. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 12 120–12 145. [Online]. Available: <https://aclanthology.org/2025.findings-emnlp.648/>
- [19] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE transactions on pattern analysis and machine intelligence*, vol. 42, no. 4, pp. 824–836, 2018.
- [20] Y. Wang, Y. Yu, K. Song, J. Lin, and Z. Shen, “When retrieval succeeds and fails: Rethinking retrieval-augmented generation for llms,” 2025. [Online]. Available: <https://arxiv.org/abs/2510.09106>
- [21] T. Chen, H. Wang, S. Chen, W. Yu, K. Ma, X. Zhao, H. Zhang, and D. Yu, “Dense X retrieval: What retrieval granularity should we use?” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, Eds. Miami, Florida, USA: Association for

- Computational Linguistics, Nov. 2024, pp. 15 159–15 177. [Online]. Available: <https://aclanthology.org/2024.emnlp-main.845/>
- [22] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. O. Ness, and J. Larson, “From local to global: A graph rag approach to query-focused summarization,” 2025. [Online]. Available: <https://arxiv.org/abs/2404.16130>
 - [23] Z. Guo, L. Xia, Y. Yu, T. Ao, and C. Huang, “LightRAG: Simple and fast retrieval-augmented generation,” in *Findings of the Association for Computational Linguistics: EMNLP 2025*, C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, Eds. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 10 746–10 761. [Online]. Available: <https://aclanthology.org/2025.findings-emnlp.568/>
 - [24] Y. Wang, H. Li, F. Teng, and L. Chen, “Arag: Advanced graph-based retrieval-augmented generation for llms,” 2026. [Online]. Available: <https://arxiv.org/abs/2511.05549>
 - [25] P. Yang, X. Li, Z. Hu, J. Wang, J. Yin, H. Wang, L. He, S. Yang, S. Wang, Y. Huang *et al.*, “Heterag: A heterogeneous retrieval-augmented generation framework with decoupled knowledge representations,” *arXiv preprint arXiv:2504.10529*, 2025.
 - [26] Y. Bai, X. Lv, J. Zhang, H. Lyu, J. Tang, Z. Huang, Z. Du, X. Liu, A. Zeng, L. Hou, Y. Dong, J. Tang, and J. Li, “Longbench: A bilingual, multitask benchmark for long context understanding,” 2024. [Online]. Available: <https://arxiv.org/abs/2308.14508>
 - [27] T. Kočíský, J. Schwarz, P. Blunsom, C. Dyer, K. M. Hermann, G. Melis, and E. Grefenstette, “The NarrativeQA reading comprehension challenge,” *Transactions of the Association for Computational Linguistics*, vol. 6, pp. 317–328, 2018. [Online]. Available: <https://aclanthology.org/Q18-1023/>
 - [28] P. Dasigi, K. Lo, I. Beltagy, A. Cohan, N. A. Smith, and M. Gardner, “A dataset of information-seeking questions and answers anchored in research papers,” in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, K. Toutanova, A. Rumshisky, L. Zettlemoyer, D. Hakkani-Tur, I. Beltagy, S. Bethard, R. Cotterell, T. Chakraborty, and Y. Zhou, Eds. Online: Association for Computational Linguistics, Jun. 2021, pp. 4599–4610. [Online]. Available: <https://aclanthology.org/2021.naacl-main.365/>
 - [29] X. Ho, A.-K. Duong Nguyen, S. Sugawara, and A. Aizawa, “Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps,” in *Proceedings of the 28th International Conference on Computational Linguistics*, D. Scott, N. Bel, and C. Zong, Eds. Barcelona, Spain (Online): International Committee on Computational Linguistics, Dec. 2020, pp. 6609–6625. [Online]. Available: <https://aclanthology.org/2020.coling-main.580/>
 - [30] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, “Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation,” 2024.
 - [31] Q. Team, “Qwen3 technical report,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.09388>
 - [32] DeepSeek-AI, “Deepseek-v3.2: Pushing the frontier of open large language models,” 2025. [Online]. Available: <https://arxiv.org/abs/2512.02556>
 - [33] I. Habernal and I. Gurevych, “Argumentation mining in user-generated web discourse,” *Computational linguistics*, vol. 43, no. 1, pp. 125–179, 2017.
 - [34] G. Team, “Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities,” 2025. [Online]. Available: <https://arxiv.org/abs/2507.06261>